Swarup Project Roadmap

Graph Attention Networks for Securing Social IoT from Botnet Propagation

Student: Swarup

December 9, 2025

Introduction

This 8-week plan (four 2-week milestones) guides the implementation of a Graph Attention Network (GAT) based framework for securing Social Internet of Things (Social IoT) networks against botnet propagation. The core idea is to learn to *intelligently prune and retain edges* in the Social IoT graph so as to prevent or slow botnet spread while preserving the network's core task structure (for example, sensing, actuation, or service delivery).

The project will:

- Build a Social IoT graph abstraction and a botnet propagation simulator.
- Train a GAT model to score edge importance under normal and attack conditions.
- Integrate Reinforcement Learning (RL) to learn an edge-pruning policy on top of GAT outputs.
- Use Explainable AI (XAI) techniques so that pruning decisions are accompanied by transparent justifications.

Work will start from standard graph neural network libraries (for example, PyTorch Geometric or Deep Graph Library) and public IoT or botnet datasets, and then adapt them to a Social IoT setting with explicit graph structure and attack simulations.

Social IoT Graph and Botnet Propagation Model

Consider a Social IoT network represented as a directed graph

$$G = (V, E),$$

where:

- V is the set of nodes, each representing a physical IoT device (sensor, actuator, gateway), a user, or a cloud/edge service.
- $E \subseteq V \times V$ is the set of edges, representing communication, trust, or social relationships (for example, "device follows device", "user-to-device control", "service invocation").

Each node $v \in V$ is associated with a feature vector $x_v \in \mathbb{R}^d$ capturing:

- Static properties: device type, vendor, capabilities.
- Topological features: degree, clustering coefficient, centrality scores.
- Traffic and security statistics: average packet rate, anomaly scores, past alerts.

Each edge $(u, v) \in E$ may also have features $e_{uv} \in \mathbb{R}^{d_e}$, such as protocol type, average bandwidth, latency, or historical reliability.

Botnet Propagation State

We model botnet infection as a dynamic process on the graph. At discrete time t, each node v has an infection state

$$z_v(t) \in \{0, 1\},$$

where $z_v(t) = 0$ denotes a benign node and $z_v(t) = 1$ denotes a botnet-compromised node.

A simple baseline propagation model can be:

- If $z_u(t) = 1$ and there is an edge $(u, v) \in E$, then node v becomes infected at time $t + 1$ with probability

  $$p_{uv} = \sigma\big(\beta^\top \phi(x_u, x_v, e_{uv})\big),$$

  where:
  - $\phi(\cdot)$ is a feature engineering function (for example, concatenation of node and edge features).
  - β is a learnable or manually chosen parameter vector.
  - $\sigma(\cdot)$ is a sigmoid function mapping to $(0, 1)$.
- Recovery or containment mechanisms (for example, patching or quarantine) can be added via a recovery probability r_v per node.

This simple graph-based epidemic model defines how quickly a botnet can propagate under the current connectivity pattern. It also provides a ground truth simulator to evaluate the impact of edge pruning: removing or deactivating edges should slow infection while still allowing essential communication for legitimate tasks.

GAT-based Edge Scoring and Pruning

A Graph Attention Network layer computes attention coefficients on edges to learn importance weights conditioned on node and edge features.

Let x_u, x_v be the node features for nodes u and v, and let W be a learnable linear transformation. A single-head attention mechanism can be written as:

$$e_{uv} = \mathrm{LeakyReLU}\big(a^\top [W x_u \,\|\, W x_v]\big),$$

$$\alpha_{uv} = \frac{\exp(e_{uv})}{\sum_{k \in \mathcal{N}(u)} \exp(e_{uk})},$$

$$h'_u = \sigma\left(\sum_{v \in \mathcal{N}(u)} \alpha_{uv} W x_v\right),$$

where:

- $\mathcal{N}(u)$ is the neighborhood of node u.
- a is a learnable attention vector.
- $\|$ denotes concatenation.
- $\sigma(\cdot)$ is a nonlinearity.

The learned attention coefficients α_{uv} provide a natural edge importance signal. The high-level pruning pipeline is:

1. Train a GAT model to perform a core graph task, such as node classification (benign versus suspicious), link prediction, or traffic anomaly detection.
2. Use learned attention coefficients (possibly aggregated across heads or layers) to derive a normalized edge score s_{uv}.
3. Prune or downweight edges with low scores s_{uv} according to a policy that balances:
   - Security objective: minimize botnet spread in the propagation simulator.
   - Utility objective: maintain acceptable performance on the core Social IoT tasks.

Reinforcement Learning and Explainable AI

While static thresholds on s_{uv} are simple, they may be suboptimal under varying attack patterns and dynamic network conditions. Reinforcement Learning (RL) can be used to learn a policy π that decides which edges to prune or retain.

A possible RL formulation:

- **State** s_t: features derived from the current graph, such as:
  - Global statistics (number of nodes, edges, infection ratio).
  - Aggregated GAT attention distributions.
  - Recent history of pruning decisions and resulting infection spread.
- **Action** a_t: pruning decisions for a subset of edges (for example, keep or remove, or pruning ratios per node or per attention bucket).
- **Reward** R_t: a scalar combining:
  - Negative cost of infection (for example, proportion of infected nodes).
  - Penalty for disrupting the core task (for example, drop in classification accuracy or service availability).
  - Complexity or overhead penalties (for example, number of edges removed).

Explainable AI (XAI) is integrated at two levels:

1. **GAT-level explanations**: use attention weights, saliency on node features, or gradient-based attribution to explain why specific edges are assigned high or low importance.
2. **RL policy explanations**: provide summaries of policy behavior, such as which attention ranges or node types are most likely to be pruned, and counterfactual analyses (for example, what would have happened if an edge had not been pruned).

The overall goal is to produce *transparent justifications* for each pruning decision, suitable for security analysts or system operators.

Objectives

1. **Social IoT graph and data:** Construct a Social IoT graph dataset with node and edge features representative of real deployments, including normal traffic and botnet-infected scenarios (either from public datasets or a synthetic generator).
2. **Botnet propagation simulator:** Implement a configurable graph-based botnet propagation model and define evaluation metrics (for example, time to infect a fixed fraction of nodes, steady-state infection rate).
3. **GAT edge importance and static pruning:** Train a GAT model for a chosen core task, extract attention-based edge importance scores, and study static pruning strategies

(thresholding, top-k retention, structured pruning).

4. **RL-based adaptive pruning:** Formulate and implement an RL environment where an agent learns a pruning policy that optimizes security and utility trade-offs.
5. **Explainable pruning decisions:** Integrate XAI methods to generate human-interpretable explanations of edge pruning decisions at both the GAT and RL levels.
6. **Deliverables:** Produce clean, well-documented Python code, scripts to reproduce experiments, plots and tables summarizing trade-offs, and a concise report or slide deck describing methods and results.

Milestones & Timeline (8 Weeks)

Milestone 1 (Weeks 1–2): Baseline Social IoT Graph and Botnet Model

Tasks

1. Set up a fresh Python environment; install and test key libraries:
   - Core: NumPy, pandas, matplotlib or seaborn (for plotting).
   - Graph learning: PyTorch, PyTorch Geometric or Deep Graph Library.
   - RL: a baseline library if used (for example, stable baselines or a simple custom implementation).
2. Select or construct a Social IoT dataset:
   - Option A: Map an existing IoT or network intrusion dataset to a graph (devices as nodes, flows as edges).
   - Option B: Design a synthetic Social IoT generator that creates device types, social relationships, and communication patterns.
3. Define node and edge features:
   - Node features: device type encoding, degree-based features, simple traffic statistics.
   - Edge features: frequency of communication, protocol indicators, average packet size.
4. Implement the botnet propagation simulator:
   - Initialize a subset of nodes as infected seeds.
   - Propagate infection in discrete time using the probabilistic model described above.
   - Log infection curves and core metrics over time.
5. Create basic visualization scripts:
   - Infection fraction over time.
   - Degree distribution and basic graph statistics.

Acceptance

- Social IoT graph construction code runs without errors and produces at least one non-trivial network instance.
- Botnet propagation simulation runs and generates infection curves for different initial conditions.
- Data and scripts are organized (for example, graph objects saved, simple README explaining how to generate the baseline graphs and infection plots).

Milestone 2 (Weeks 3–4): GAT for Edge Importance and Static Pruning

Tasks

1. Define a core supervised or semi-supervised task on the Social IoT graph, for example:
   - Node classification (benign versus compromised or vulnerable).
   - Service availability prediction or device role classification.
2. Implement a baseline GAT architecture:
   - 1–2 GAT layers with multi-head attention.
   - Suitable readout and classification head (for example, softmax for node labels).
3. Train the GAT model on the baseline graph:
   - Split nodes into train, validation, and test sets.
   - Log training and validation loss and accuracy versus epochs.
   - Ensure stable convergence with appropriate hyperparameters.
4. Extract attention-based edge importance:
   - For each edge (u, v), compute an aggregated importance score s_{uv} from the attention coefficients (for example, average or maximum over heads and layers).
   - Analyze the distribution of s_{uv} values and correlate them with graph properties (for example, high-degree hubs, central nodes).
5. Implement static pruning strategies:
   - Threshold-based pruning: remove edges with s_{uv} below a chosen percentile.
   - Top-k pruning per node: retain only the k most important edges in each node's neighborhood.
6. Evaluate trade-offs:
   - Measure botnet spread metrics on the pruned graph versus the original graph.
   - Measure core task performance (for example, classification accuracy) before and after pruning.

Acceptance

- GAT training runs reproducibly and achieves reasonable performance on the chosen task.
- Static pruning scripts execute without errors and log both security (infection metrics) and utility (task performance) results.
- At least one figure or table clearly showing the trade-off between pruning aggressiveness, botnet suppression, and task performance.

Milestone 3 (Weeks 5–6): RL-based Adaptive Edge Pruning with XAI Hooks

Tasks

1. Formalize the RL environment:
   - Specify state representation using GAT outputs and global graph metrics.
   - Define action space (for example, discrete pruning levels per attention bin or per region of the graph).
   - Design a reward function balancing infection suppression, task performance, and pruning cost.
2. Implement the RL loop:
   - Choose an RL algorithm (for example, a simple Deep Q Network or policy gradient method).
   - Integrate the botnet simulator and GAT edge scoring within the environment step function.
   - Run multiple episodes to train the policy.
3. Integrate Explainable AI for pruning decisions:

- Log attention distributions and node features for edges that are pruned or retained by the RL policy.
- Generate simple explanation artifacts, such as:
  - Lists of edges most frequently pruned and their typical characteristics.
  - Per-node summaries explaining why certain neighbors are removed.

4. Evaluate adaptive pruning:
   - Compare RL-based pruning versus static thresholds in terms of infection metrics and task performance.
   - Test robustness under different simulated attack scenarios (for example, varying number of initial infected nodes or infection probabilities).

Acceptance

- RL environment and training loop run end-to-end without errors.
- Learned policy demonstrates at least some improvement or meaningful difference versus simple static pruning baselines.
- Basic XAI summaries are available, showing which kinds of edges are typically pruned and why.

Milestone 4 (Weeks 7–8): Polishing, Ablations, and Deliverables

Tasks

1. Perform ablation studies:
   - Compare different GAT architectures (number of layers, heads).
   - Compare different RL reward weightings (security versus utility).
   - Explore the impact of noisy or incomplete node features.
2. Stress tests:
   - Vary network size and density to check scalability.
   - Vary botnet aggressiveness (higher infection probabilities) and observe how the RL policy adapts.
3. Final figures and tables:
   - Infection curves under no pruning, static pruning, and RL-based pruning.
   - Core task performance versus pruning level.
   - Example explanation plots or tables for pruning decisions.
4. Documentation and packaging:
   - Clean up code, ensure all scripts can be run via a small number of commands.
   - Write a concise technical note or mini-report summarizing the problem setting, methods, and key results.
   - Prepare a short slide deck for presentation.

Acceptance

- All experiments can be reproduced via documented scripts and configuration files.
- A clear narrative is available (report and slides) explaining how GAT, RL, and XAI interact to secure Social IoT networks against botnet propagation.
- Repository is in a presentable state for sharing with supervisors or collaborators.

Immediate Action Items (Week 1)

1. Create a version-controlled repository (for example, "gat-social-iot-swarup") with a minimal README describing the project goal and environment setup instructions.
2. Set up the Python environment and verify installation of graph learning and RL libraries by running a small example (for example, a toy GAT on a citation graph or a small synthetic graph).
3. Implement the first Social IoT graph generator or data loader, and run the baseline botnet propagation simulator to produce initial infection curves and basic plots.